CLOUD COMPUTING

Assignment

Name:-Eman Tanveer
Reg No:-FA24B1-SE-053
Submitted To:-Sir Waqas
Department:-Software Engineering
Section:-A

Eman Tanveer

Assignment 01: Git, Gitea, GitHub, Git LFS, and GitHub Pages

Summary

This solution report documents the complete practical execution of **Assignment 01** for Cloud Computing. The lab work demonstrates essential DevOps and Cloud infrastructure concepts, including containerized service hosting, multi-remote Git management, binary large object handling with Git LFS, and public static web hosting via GitHub Pages.

Task 1: Self-Hosted Gitea Service Setup

1.1Concepts

- **What is Gitea?** Gitea is a fast, lightweight, self-hosted Git service (similar to GitHub). Hosting it on an Ubuntu server provides full control over private repositories and code infrastructure.
- **Why Docker Compose?** Docker Compose allows us to define and run multi-container Docker applications easily using a docker-compose.yml file. It ensures the Gitea container runs consistently without manual dependencies.

1.2 Step-by-Step Procedure

1. Created a dedicated working directory ~/gitea on the Ubuntu server.
2. Configured docker-compose.yml specifying the Gitea image, port mappings (3000:3000 for Web UI, 2222:22 for SSH), and volume storage.
3. Started the container using docker compose up -d and confirmed container health.
4. Accessed http://localhost:3000, completed initial admin configuration, and created the public repository Assignment01.
5. Pushed initial project files from the Ubuntu server to Gitea.

Screenshots:-

1. Gitea Container Status

- **Filename:** task1_gitea_running.png
- **What to show:** Terminal screen running docker ps showing the active Gitea container and exposed port 3000.

```
ompose ps
curl -sS -o /dev/null -w "Geman-tanveer123@ubuntu:~/Gitea$ sudo docker compose ps
tea HTTP status: %{http_code}\n" http://127.0.0.1:3000/
ip addr[sudo: authenticate] Password:
sudo: Authentication failed, try again.
[sudo: authenticate] Password:
NAME       IMAGE                COMMAND                  SERVICE   CREATED          STATUS          PORTS
gitea      gitea/gitea:latest   "/usr/bin/entrypoint…"   gitea     21 minutes ago   Up 7 minutes    22/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
gitea_db   postgres:17-alpine   "docker-entrypoint.s…"   db        21 minutes ago   Up 21 minutes   5432/tcp
eman-tanveer123@ubuntu:~/Gitea$ cd ~/Gitea
docker compose ps
curl -sS -o /dev/null -w "Gitea HTTP status: %{http_code}\neman-tanveer123@ubuntu:~/Gitea$ docker compose ps
 http://127.0.0.1:3000/
ip addrpermission denied while trying to connect to the docker API at unix:///var/run/docker.sock
eman-tanveer123@ubuntu:~/Gitea$ curl -sS -o /dev/null -w "Gitea HTTP status: %{http_code}\n" http://127.0.0.1:3000/
Gitea HTTP status: 200
eman-tanveer123@ubuntu:~/Gitea$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:38:2a:ef brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29382aef
    inet 192.168.15.130/24 metric 100 brd 192.168.15.255 scope global dynamic ens33
       valid_lft 1383sec preferred_lft 1383sec
    inet6 fe80::20c:29ff:fe38:2aef/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c6:26:c9:38:fb:46 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-e58f07156a82: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 8e:74:32:38:11:84 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-e58f07156a82
       valid_lft forever preferred_lft forever
    inet6 fe80::8c74:32ff:fe38:1184/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
5: veth5c132940@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-e58f07156a82 state UP group default
    link/ether 72:5c:b2:0d:31:54 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::7ca6:90ff:fee1:5364/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
8: veth866c3ac@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-e58f07156a82 state UP group default
    link/ether 66:cf:9d:ff:f3:78 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::64ef:9dff:feff:f378/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
eman-tanveer123@ubuntu:~/Gitea$ _
```

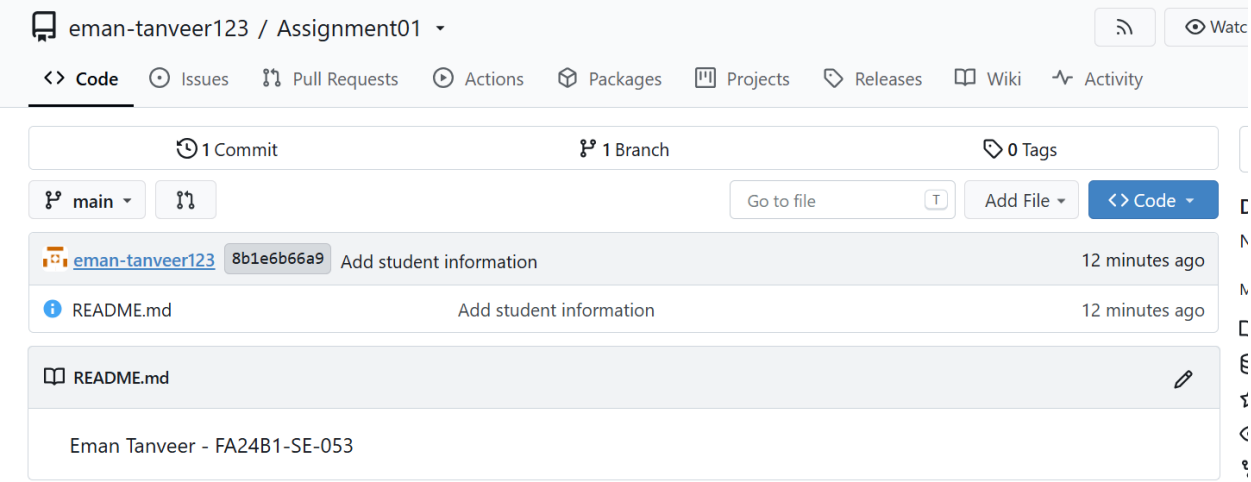
2. Initial Gitea Push Output

- **Filename:** task1_gitea_push.png
- **What to show:** Terminal screen showing the successful git push output to the Gitea remote URL.

```
 create mode 100644 README.md
eman-tanveer123@ubuntu:~/Assignment01$ git remote add gitea http://eman-tanveer123@127.0.0.1:3000/eman-tanveer123/Assignment01.git
itea maineman-tanveer123@ubuntu:~/Assignment01$ git remote -v
gitea   http://eman-tanveer123@127.0.0.1:3000/eman-tanveer123/Assignment01.git (fetch)
gitea   http://eman-tanveer123@127.0.0.1:3000/eman-tanveer123/Assignment01.git (push)
eman-tanveer123@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://eman-tanveer123@127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 256 bytes | 256.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/eman-tanveer123/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
eman-tanveer123@ubuntu:~/Assignment01$
```

3. Gitea Web Repository

- **Filename:** task1_gitea_repository.png
- **What to show:** Browser window showing the Gitea web interface with the Assignment01 repository page and files.



Task 2: Multi-Remote Repository Management

2.1Concepts

- **What are Git Remotes?** A Git remote is a common repository stored on a server. Git allows adding multiple remotes to a single local repository.
- **Why Dual Remotes?** Setting up dual remotes (origin for local Gitea and github for GitHub) allows a developer to sync code across private on-premise servers and public cloud platforms using unified Git commands.

2.2 Step-by-Step Procedure

1. Configured global Git identity (user.name and user.email) on the Ubuntu server.
2. Initialized the local repository in ~/CC/Assignments/Assignment01.
3. Added the primary Gitea remote (origin) and secondary GitHub remote (github) using git remote add.
4. Staged, committed, and pushed changes to both Gitea and GitHub remotes using personal access tokens.

Screenshots:-

4. Remote Configuration Verification

- **Filename:** task2_remotes.png

- **What to show:** Terminal output of git remote -v displaying both origin (Gitea) and github (GitHub) URLs.



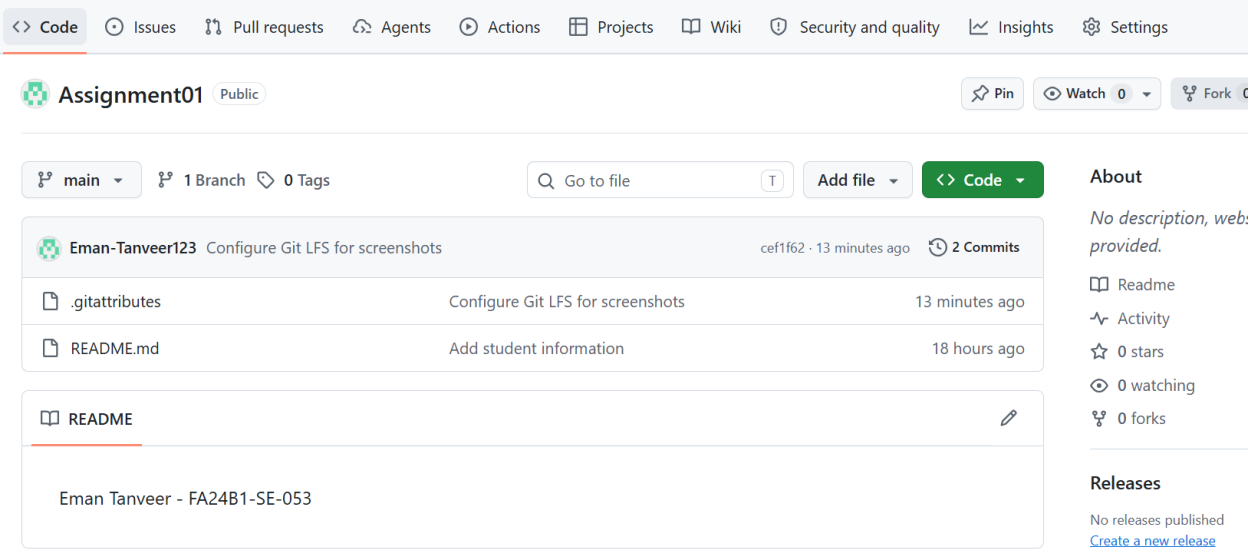
5. GitHub Push Output

- **Filename:** task2_github_push.png
- **What to show:** Terminal output showing successful git push github main command execution.



6. GitHub Web Repository

- **Filename:** task2_github_repository.png
- **What to show:** Browser page showing your GitHub Assignment01 public repository with pushed files visible.



Task 3: Git Large File Storage (Git LFS)

3.1Concepts

- **The Large File Problem:** Regular Git is designed for text source code. Tracking large binary files directly bloats repository history and causes slow download/upload performance.
- **How Git LFS Works:** Git LFS replaces large files inside the Git repository with tiny pointer files. The actual heavy binary data is stored on a separate LFS server, keeping your Git repository fast and lightweight.

3.2 Step-by-Step Procedure

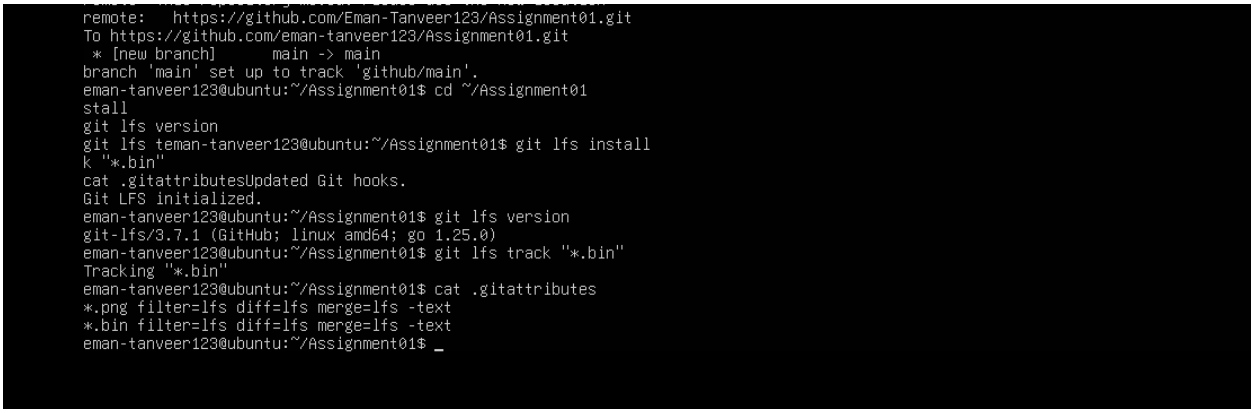
1. Installed git-lfs on Ubuntu and initialized it using git lfs install.
2. Configured tracking rules for specific large extensions or file names using git lfs track.
3. Verified tracking configuration saved inside the .gitattributes file.
4. Added 3 large binary files, committed them, and verified LFS tracking status.
5. Pushed the repository including LFS binary pointers to the remotes.

Screenshots:-

7. Git LFS Initialization & Tracking

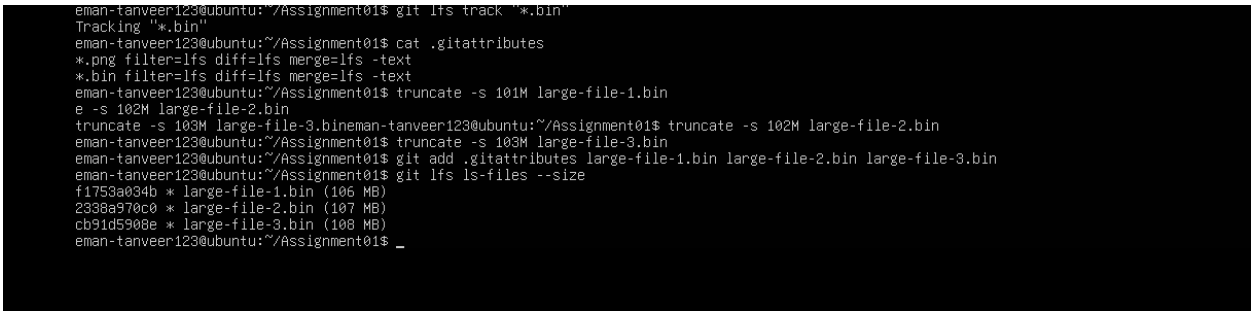
- **Filename:** task3_lfs_setup.png

- **What to show:** Terminal screen showing git lfs install, git lfs track commands, and the contents of .gitattributes.



8. Tracked LFS Binary Files

- **Filename:** task3_lfs_files.png
- **What to show:** Terminal output of git lfs ls-files displaying the three large files tracked by LFS along with their hashes.



9. Git LFS Push Output

- **Filename:** task3_lfs_push.png
- **What to show:** Terminal screen showing git push output uploading Git LFS objects.

```
eman-tanveer123@ubuntu:~/Assignment01$ git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
eman-tanveer123@ubuntu:~/Assignment01$ git lfs ls-files --size
f1753a034b * large-file-1.bin (106 MB)
2338a970c0 * large-file-2.bin (107 MB)
cb91d5908e * large-file-3.bin (108 MB)
eman-tanveer123@ubuntu:~/Assignment01$ git commit -m "Add three files using Git LFS"
[main 1a35115] Add three files using Git LFS
 4 files changed, 10 insertions(+)
 create mode 100644 large-file-1.bin
 create mode 100644 large-file-2.bin
 create mode 100644 large-file-3.bin
eman-tanveer123@ubuntu:~/Assignment01$ git push github main
Uploading LFS objects: 100% (3/3), 321 MB | 470 KB/s, done.
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 804 bytes | 114.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:   https://github.com/Eman-Tanveer123/Assignment01.git
To https://github.com/eman-tanveer123/Assignment01.git
   cef1f62..1a35115  main -> main
eman-tanveer123@ubuntu:~/Assignment01$ _
```

Task 4: Portfolio Deployment via GitHub Pages

4.1Concepts

- **What is GitHub Pages?** GitHub Pages is a static web hosting service that serves HTML, CSS, and JavaScript files directly from a GitHub repository to the public internet.
- **User Page Convention:** By naming the repository <username>.github.io, GitHub automatically publishes the website live at https://<username>.github.io/.

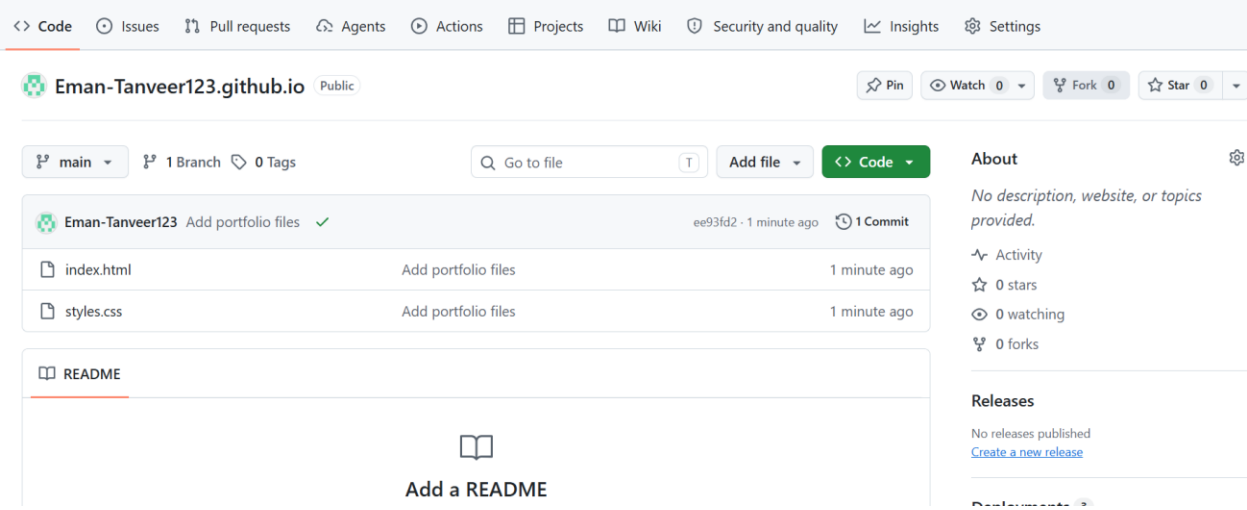
4.2 Step-by-Step Procedure

1. Created a public repository named eman-tanveer123.github.io.
2. Developed portfolio web files (index.html and styles.css) containing **About Me**, **Education**, **Skills**, and **Projects** sections.
3. Pushed the portfolio code to the main branch.
4. Verified deployment under Repository Settings $\rightarrow$ Pages.
5. Loaded and verified the active live site at https://eman-tanveer123.github.io/.

Screenshots:-

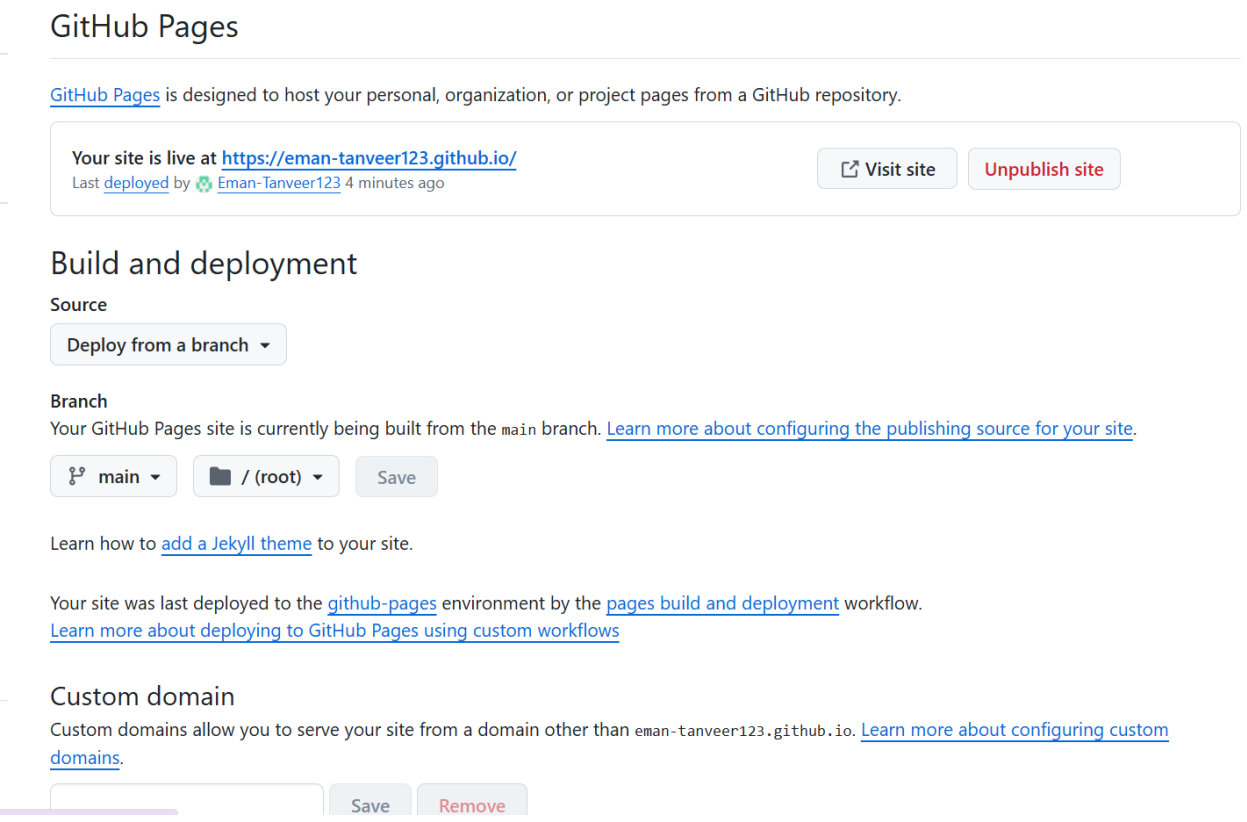
10. Portfolio Source Repository

- **Filename:** task4_pages_repository.png
- **What to show:** Browser window showing the eman-tanveer123.github.io repository on GitHub with index.html and styles.css.



11. GitHub Pages Deployment Status

- **Filename:** task4_pages_deployment.png
- **What to show:** Browser page of Repository Settings $\rightarrow$ Pages showing "Your site is live at https://eman-tanveer123.github.io/".



12. Live Portfolio Verification

- **Filename:** task4_portfolio_live.png
- **What to show:** Browser page of the active website at https://eman-tanveer123.github.io/ displaying full sections (About Me, Education, Skills, Projects).

